



UNIVERSIDAD
CATÓLICA
SEDES SAPIENTIAE

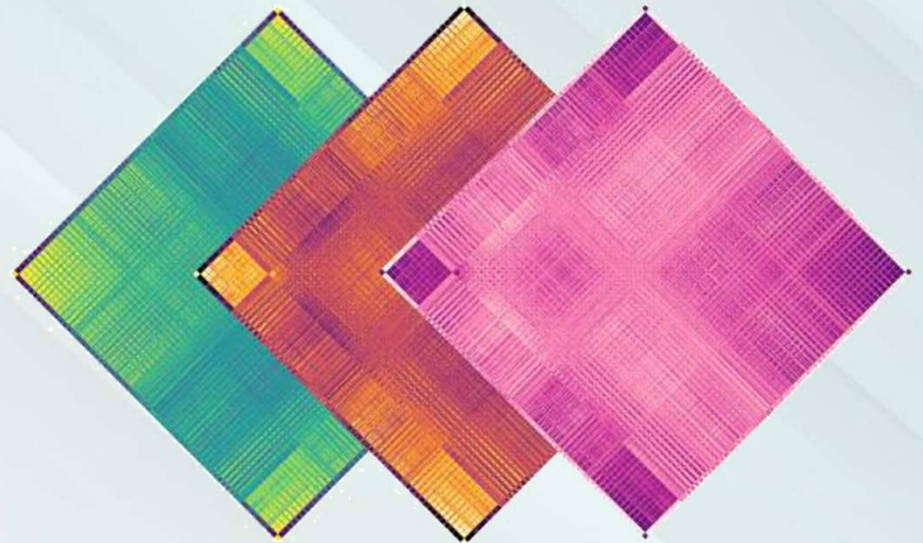
Inteligencia Artificial

Ing. Juancarlos Santana Huamán
jsantana@ucss.edu.pe



Redes de Hopfield: máquinas de memoria neuronal

- Las redes neuronales recurrentes capaces de almacenar y recuperar múltiples memorias.
- Comenzaremos con una descripción conceptual detallada y luego pasaremos a una implementación de redes de Hopfield desde cero en Python.





Redes de Hopfield: máquinas de memoria neuronal

- En cada momento de tu vida, ondas de actividad se propagan por las redes de tu cerebro. Miles de millones de señales se armonizan y oscilan continuamente; estas redes constituyen la arquitectura funcional que sustenta todo lo que existe. Son tu amor, tu estrés, tu canción favorita, tus esperanzas y sueños. Toda tu percepción del ser y tu experiencia se forman a partir del comportamiento dinámico de estas redes, que se mantienen estables gracias a sistemas de memoria que se adaptan constantemente para representar mejor tu lugar en un entorno en constante cambio.
- El gran alcance e integración de las redes en el cerebro humano hacen que sea increíblemente difícil estudiar cómo, dónde o incluso *si* ocurre la computación en las formas con las que estamos familiarizados. Existe evidencia de procesamiento de información a niveles de proteínas moduladoras en células individuales, microcircuitos corticales y redes funcionales de todo el cerebro , entre otros. La comprensión experimental del cerebro avanza lentamente.
- Afortunadamente, ingenieros han inventado o descubierto algoritmos que pueden, en parte, modelar aspectos de estas redes. Ningún modelo único encapsulará jamás la complejidad y el comportamiento absolutos del cerebro humano, pero estas herramientas brindan a los estudiantes de tales sistemas una ventana conveniente para observar las formas en que la información podría ser computada, y en última instancia representada, dentro de la actividad de una red distribuida.





ANTECEDENTES CONCEPTUALES

- Las redes de Hopfield son una forma atractiva de Redes Neuronales Artificiales Recurrentes (RNN), descritas por primera vez por John Hopfield en su artículo de 1982 titulado " Redes neuronales y sistemas físicos con capacidades computacionales colectivas emergentes ". Cabe destacar que las redes de Hopfield fueron el primer ejemplo de redes neuronales asociativas: arquitecturas de RNN capaces de producir una memoria asociativa emergente.
- La memoria asociativa, o memoria direccionable por contenido, es un sistema en el que la evocación de un recuerdo se inicia mediante la asociabilidad de un patrón de entrada con uno memorizado. En otras palabras, la memoria asociativa permite la recuperación y finalización de un recuerdo utilizando solo una parte incompleta o con ruido.
- Por ejemplo, una persona podría escuchar una canción que le gusta y revivir el recuerdo de la primera vez que la escuchó. El contexto, las personas, el entorno y las emociones de ese recuerdo pueden recuperarse mediante la exposición posterior a solo una parte del estímulo original: la canción. Estas características de las redes de Hopfield (HN) las convirtieron en buenas candidatas para los primeros modelos computacionales de la memoria asociativa humana y marcaron el comienzo de una nueva era en la computación y el modelado neuronal.





ANTECEDENTES CONCEPTUALES

- La arquitectura de red única de Hopfield se basó en el modelo Ising, un modelo físico que explica el comportamiento emergente de los campos magnéticos producidos por materiales ferromagnéticos. El modelo generalmente se describe como un gráfico plano, donde los nodos representan momentos dipolares magnéticos de arreglos regulares que se repiten. Cada nodo puede ocupar uno de dos estados (es decir, espín hacia arriba o espín hacia abajo, +1 o -1), y el acuerdo de estados entre nodos adyacentes es energéticamente favorable. A medida que cada dipolo 'voltea', o evoluciona para encontrar Mínimos energéticos locales: todo el material tiende hacia un estado estacionario o mínimo energético global. Las redes neuronales de alta energía (HN) pueden concebirse de forma similar, donde la red se representa como un grafo 2D y cada nodo (o neurona) ocupa uno de dos estados: activo o inactivo. Al igual que en los modelos de Ising, el comportamiento dinámico de una HN se determina en última instancia según la noción metafórica de «energía del sistema», y la red converge a un estado de «mínimo energético», que en el caso de una red aprendida resulta ser una memoria.
- Para comprender el diseño y la función de una red de Hopfield, es útil desglosar las cualidades y los atributos de la unidad/neurona individual.





Red Neuronal

Cada neurona de la red tiene tres cualidades a considerar:

- **Conexiones con otras neuronas:** cada neurona de la red está conectada a todas las demás, y cada conexión tiene una fuerza o peso único, similar a la fuerza de una sinapsis. Estas fuerzas de conexión se almacenan en una matriz de pesos.
- **Activación:** se calcula mediante la entrada neta de otras neuronas y sus respectivos pesos de conexión, de forma similar al potencial de membrana de una neurona. La activación adquiere un único valor escalar.
- **Estado bipolar:** esta es la salida de la neurona, calculada mediante su activación y una función de umbralización, análoga a su estado de activación. En este caso, -1 y +1.





Red Hopfield

- La información, o los recuerdos de una red de Hopfield, se almacenan en la fuerza de las conexiones entre neuronas, de forma muy similar a como se cree que se almacena la información en el cerebro según los modelos de memoria a largo plazo. Las conexiones y sus respectivos pesos afectan directamente la actividad de la red con cada generación y, en última instancia, determinan el estado al que converge.
- Se puede imaginar el peso entre dos neuronas, la neurona a y la neurona b, como la medida en que la salida de la neurona a contribuirá a la activación de la neurona b, y viceversa. Es importante destacar que las conexiones entre neuronas en una HN son simétricas. Esto significa que si la neurona a está conectada a la neurona b con una fuerza de +1, la conexión de la neurona b a la neurona a también es +1. Podemos almacenar los pesos de una red con n neuronas en una matriz simétrica cuadrada de dimensiones $n \times n$. Debido a que las neuronas no se reconectan entre sí, los esos en la diagonal de la matriz son efectivamente nulos.
- Es importante notar que la estructura de la red, incluyendo el número de neuronas y sus conexiones en la red, es fija. Los pesos de esas conexiones son el parámetro ajustable. Cuando hablamos de "aprendizaje de red", lo que realmente queremos decir es el ajuste fino de cada peso en la red para lograr algún resultado deseado. En el caso de una HN, la configuración del estado inicial puede considerarse como una entrada y su configuración del estado final como la salida. Dado que cada neurona se actualiza según las combinaciones lineales de sus entradas, la red completa es en realidad una función matemática masiva: un sistema dinámico que, con cada generación de actualización, toma su salida anterior como entrada actual. Los parámetros de esta función, los pesos, determinan la evolución de la red a lo largo del tiempo.





Red Hopfield

- La activación instantánea de cualquier neurona se puede calcular tomando la **suma ponderada** de las entradas de las neuronas a las que está conectada:

$$y_i = \sum_j y_j w_{ij}$$

- Donde y_i representa la activación de la neurona -i en la red, y_j representa un vector de las salidas respectivas de todas las neuronas que ingresan a la neurona -i, y w_{ij} el peso simétrico de la conexión entre las neuronas i y j .
- La activación de una neurona se utiliza para determinar el estado o salida de la neurona según una función de umbral:

$$s_i = \begin{cases} +1 & \text{if } y_i > 0 \\ -1 & \text{if } y_i < 0 \end{cases}$$

- Donde s_i representa el estado y la salida dados de una neurona i.





Red Hopfield

- A medida que se actualiza el estado de cada neurona de la red, sus contribuciones a las actividades de otras neuronas cambian; esto, a su vez, retroalimenta y modula más actividad, y así sucesivamente, hasta que la red alcanza una formación estable. Si la activación de una neurona durante una generación dada es positiva, su salida será +1 y viceversa. Otra forma de pensar esto es que una neurona dada recibe un "campo" de entradas análogo a su activación. Cada entrada que recibe la neurona se pondera y se suma para obtener una activación neta, ya sea positiva o negativa. Si el signo de ese campo difiere del signo de la salida actual de la neurona, su estado se invertirá para alinearse. Si el signo del campo de entrada coincide con el signo de la salida actual de la neurona, se mantendrá igual. Si consideramos el comportamiento de la red global según estas reglas, la calidad de la recurrencia se hace evidente; cada actualización de la red, o salida, se convierte en la entrada para el siguiente paso de tiempo.





Red Hopfield

- Ahora hemos definido las características de las unidades individuales, pero ¿cómo se traduce esto al almacenamiento distribuido y la representación de información en la red?
- Si bien cada neurona puede representar uno de dos estados en un momento dado, el estado general de la red, denominado s , puede representar una cadena de información binaria. Tomemos este patrón como ejemplo:

Pattern = [0,1,0, 1,0,1, 0,1,0]

- Este patrón es un vector binario de 1 x 9 que se puede transformar en una matriz de 3 x 3:

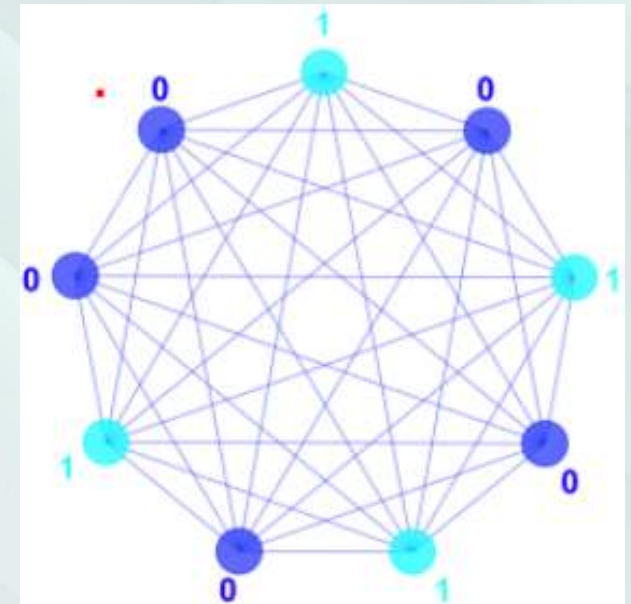
*Pattern.reshape(3,3) = [[0,1,0],
[1,0,1],
[0,1,0]]* → When drawn looks like:

0	1	0
1	0	1
0	1	0



Red Hopfield

- Una imagen es simplemente una matriz de valores numéricos de píxeles, por lo que cualquier imagen que pueda representarse con valores de píxeles que ocupen estados binarios puede representarse con precisión mediante los estados de un HN binario. Muchas formas de información pueden representarse de forma binaria/bipolar.
- Si un patrón toma la forma de una imagen, representamos los píxeles mediante neuronas individuales. Anteriormente, tenemos una matriz de píxeles de 3 x 3, por lo que necesitamos 9 neuronas para representar la imagen completa. Otra forma de visualizar esto es representar la red:





Red Hopfield

- En este gráfico de red, los nodos representan neuronas y los bordes, las conexiones entre ellas. El punto rojo indica la neurona cuyo estado representa el primer elemento, 1, en el vector de patrones. Siguiendo el estado de las neuronas en sentido horario se obtiene el resto del patrón.
- En una computadora, el estado de cada neurona se representa como los elementos individuales en un vector de 1×9 , S . En otras palabras, representamos directamente la información que intentamos almacenar utilizando los respectivos estados de las neuronas en la red:

$$S = [0,1,0,1,0,1, 0,1,0].$$

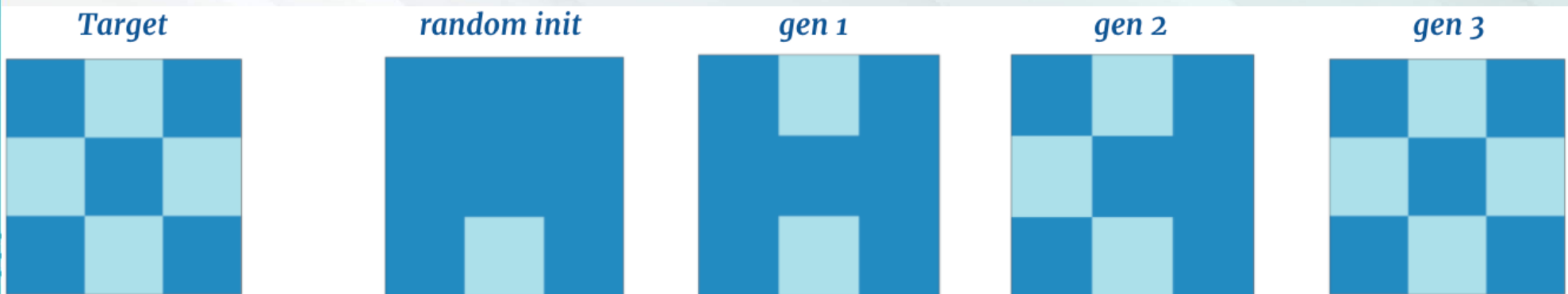
- Cualquier cadena de información arbitraria de n bits puede representarse mediante una red de n neuronas. Es la actividad combinada de las neuronas de la red la que, en conjunto, representa la información objetivo de forma distribuida. En la mayoría de las redes neuronales, los estados de las neuronas son bipolares (-1 o +1). Para simplificar, el ejemplo anterior utiliza el binario 0 o 1; de aquí en adelante, supongamos que las neuronas ocupan estados bipolares.





Red Hopfield

- Ahora que sabemos cómo se actualizan los estados de las neuronas a lo largo del tiempo, y cómo los estados en sí mismos representan información binaria, llegamos a la verdadera ~magia~ de las redes de Hopfield: su evolución hacia un patrón memorizado.
- Aquí, un poco más de información contextual resulta útil. Las redes neuronales de atracción suelen denominarse "redes atractoras" porque tienden a evolucionar hacia un estado estacionario, o son atraídas por él. Para ver esto en acción, consideremos de nuevo el patrón de ejemplo anterior. Para esta sección, supongamos que la red ya ha aprendido el patrón (profundizaremos en los detalles del aprendizaje más adelante). Supongamos que inicializamos la red en un estado aleatorio y la dejamos funcionar según las reglas de actualización descritas anteriormente, una neurona a la vez:

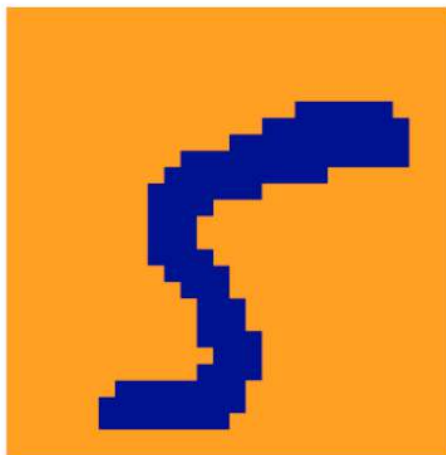




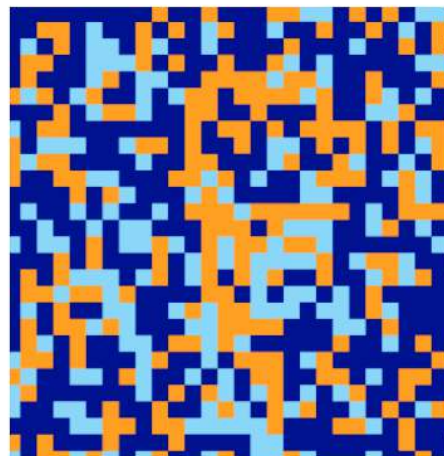
Red Hopfield

- Aquí vemos que, a pesar del estado inicializado aleatorio de la red, esta pudo restaurar la memoria objetivo en tan solo tres pasos de generación de actualizaciones: el estado general de la red se atrajo al estado de la memoria. Aumentemos un poco la resolución de nuestros patrones objetivo:

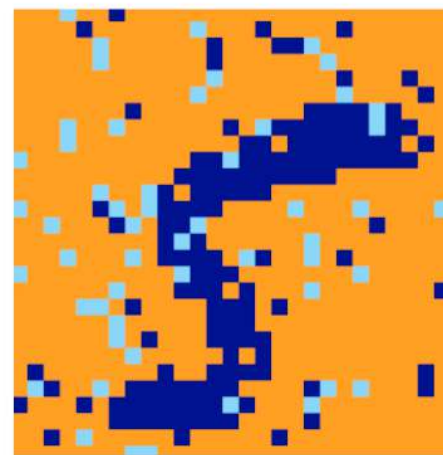
Target



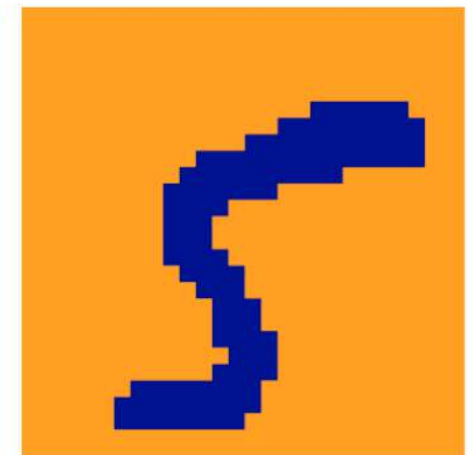
random init



gen 15



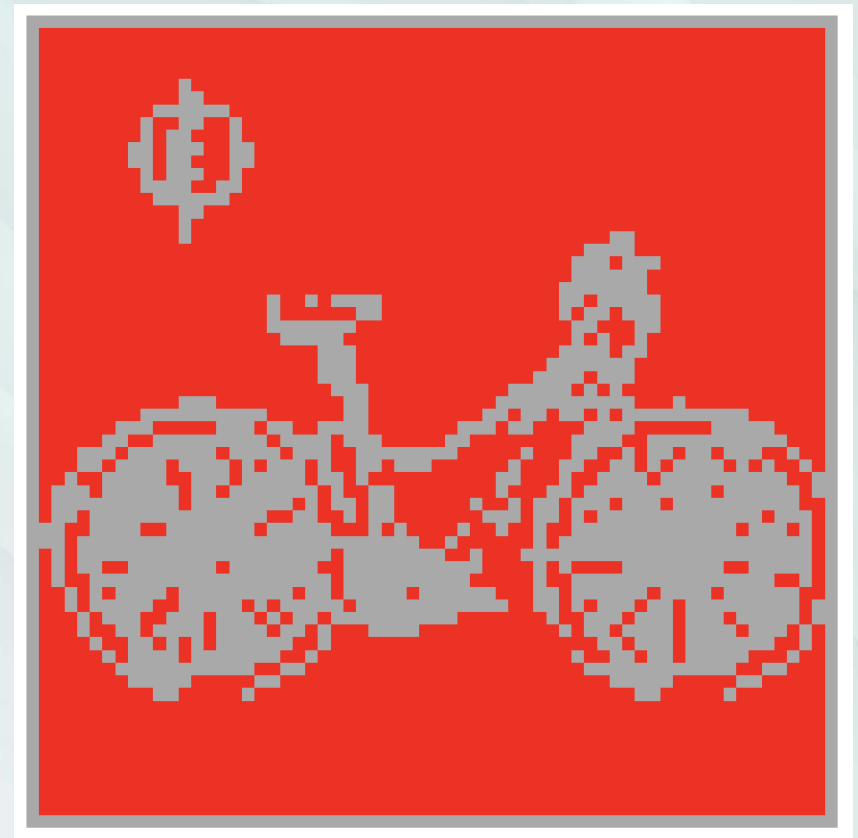
gen 40





Red Hopfield

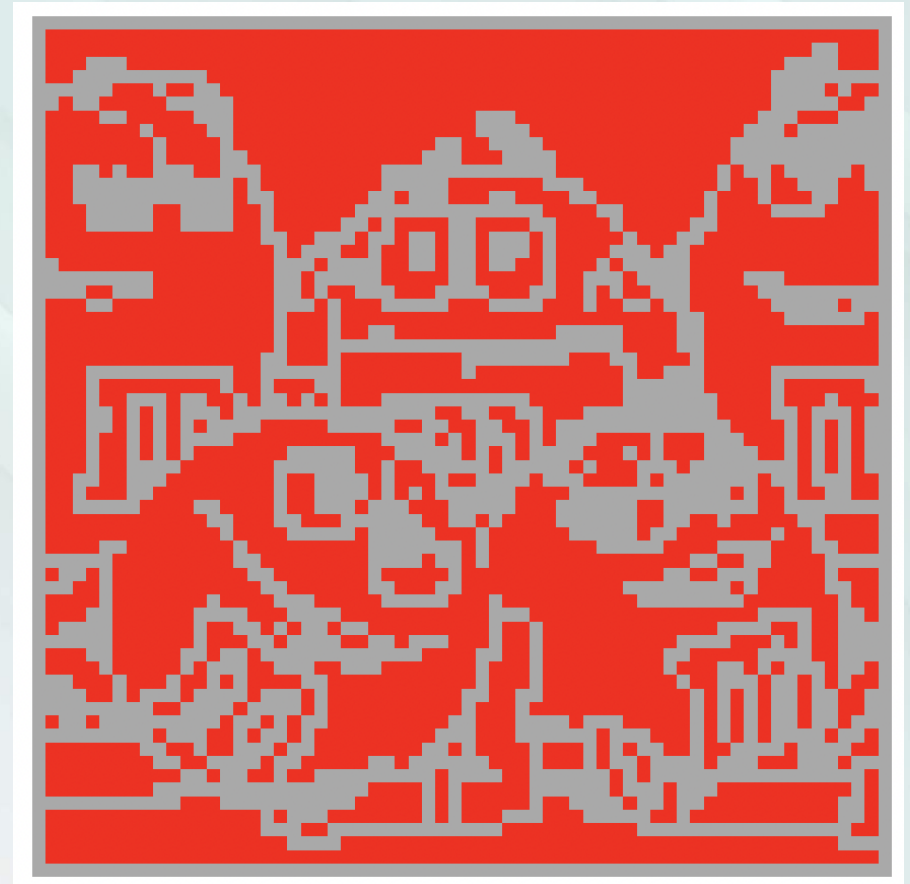
- Una vez más, la red converge lentamente a un estado estable, en este caso un "5" dibujado a mano, y no se actualiza más. Este comportamiento es significativo. A pesar de que el estado inicial de cada neurona está alterado, los pesos de las conexiones entre ellas, junto con sus respectivas activaciones y salidas, son suficientes para acercar el estado general de la red al patrón memorizado con cada actualización posterior. Cada neurona solo conoce su propio estado y las entradas entrantes, y, aun así, de la actividad colectiva de la red surge un patrón distribuido.
- Las redes configuradas de esta manera pueden almacenar hasta $\sim 0,14n$ patrones, donde n representa el número de neuronas en la red. Al almacenar varios patrones, la red convergerá al patrón más similar a su estado inicial.





UNIVERSIDAD
CATÓLICA
SEDES SAPIENTIAE

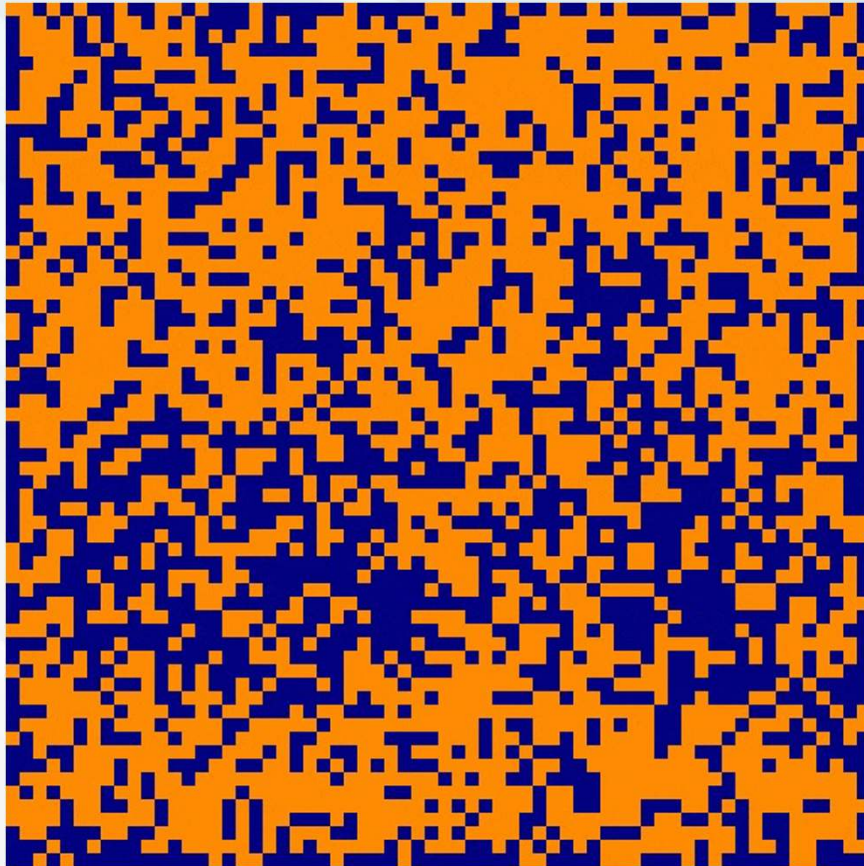
Red Hopfield





UNIVERSIDAD
CATÓLICA
SEDES SAPIENTIAE

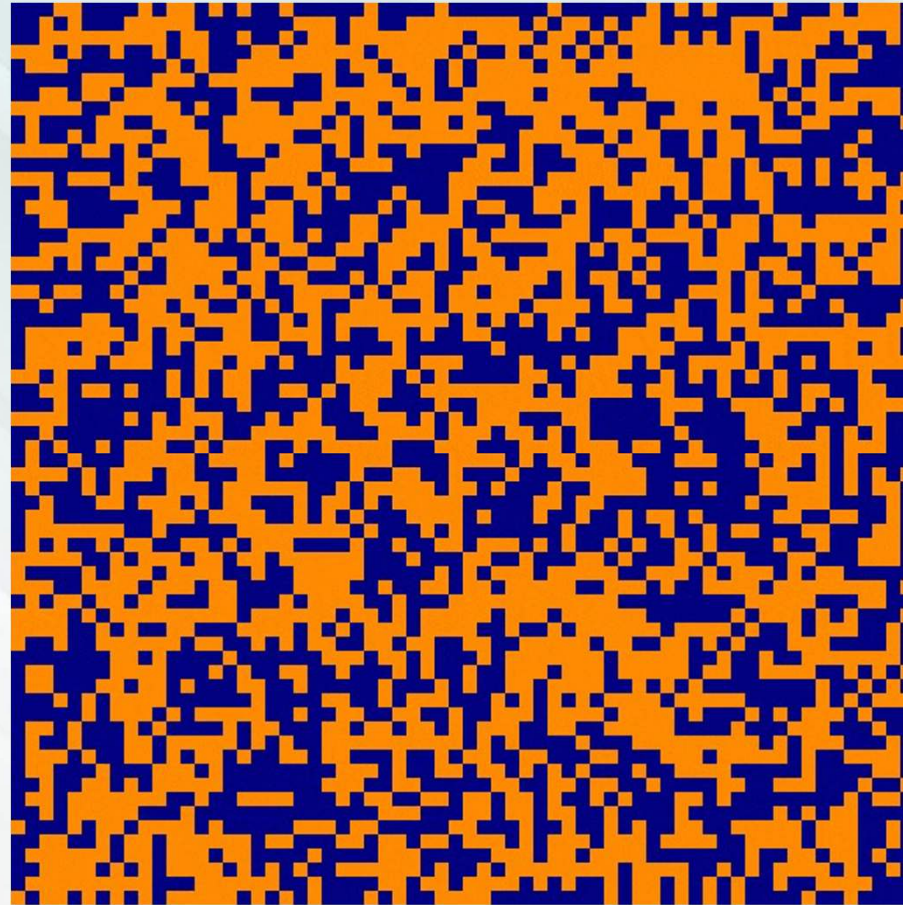
Red Hopfield





UNIVERSIDAD
CATÓLICA
SEDES SAPIENTIAE

Red Hopfield





Red Hopfield

- En el ejemplo anterior, la misma red neuronal de 4096 neuronas ha memorizado tres patrones distintos (las imágenes rojas). Cada patrón es una imagen de 64×64 píxeles aprendida por la red. Posteriormente, la red se inicializa tres veces a partir de una versión distorsionada de cada imagen. Durante cada generación de actualización, se seleccionan aleatoriamente 256 neuronas (7,2 % de la red) y se actualizan según la regla de actualización de umbral binario. Aunque cada inicialización de la red está muy distorsionada, la red restaura cada patrón respectivo a medida que su estado converge. Gracias al aprendizaje de varios patrones, la red ha desarrollado un comportamiento flexible y específico para cada estímulo.
- En estos recuerdos de ejemplo, la información que reconocemos son las relaciones espaciales simples de píxeles iluminados y apagados, es decir, las formas, que hacen que una imagen sea reconocible. Sin embargo, cualquier información que pueda almacenarse como una cadena de valores binarios puede ser memorizada y recuperada por una red neuronal artificial (HN). En una sección posterior de la publicación se abordarán las aplicaciones más avanzadas de las HN, pero para dar rienda suelta a la imaginación, volvamos al ejemplo de la introducción: Imaginemos que el estado de una red atractora integrada y de muy alta dimensión en nuestro cerebro codifica numerosos fragmentos de información sobre las circunstancias sensoriales y contextuales en las que escuchamos nuestra canción favorita por primera vez, es decir, un recuerdo.
- Con quién estabas, tu entorno inmediato, tu valencia emocional durante la experiencia, etc. Aunque estos datos codifican diferentes partes del recuerdo, podrían ser recordados simultáneamente por la misma red de memoria al escuchar posteriormente la canción sola. Este fenómeno se conoce como compleción de patrones y constituye un subárea completa dentro del estudio de la memoria humana.



Red Hopfield

- Algunos creen que el mecanismo físico está mediado por la presencia de dinámicas atractoras en el hipocampo humano; véase este extracto de " Los mecanismos para la compleción y separación de patrones en el hipocampo" , Rolls, 2013 :
- Muchas de las sinapsis (conexiones) en el hipocampo muestran una modificación asociativa como lo demuestra la potenciación a largo plazo, y esta modificación sináptica parece estar involucrada en el aprendizaje. Sobre la base de la evidencia resumida anteriormente, y otros han sugerido que la etapa CA3 actúa como una memoria de autoasociación [también llamada atractora] que permite que se formen y almacenen memorias episódicas en la red CA3, y que posteriormente la extensa conectividad recurrente permite que la recuperación de una representación completa se inicie mediante la activación de una pequeña parte de la misma representación (la señal de recuerdo).



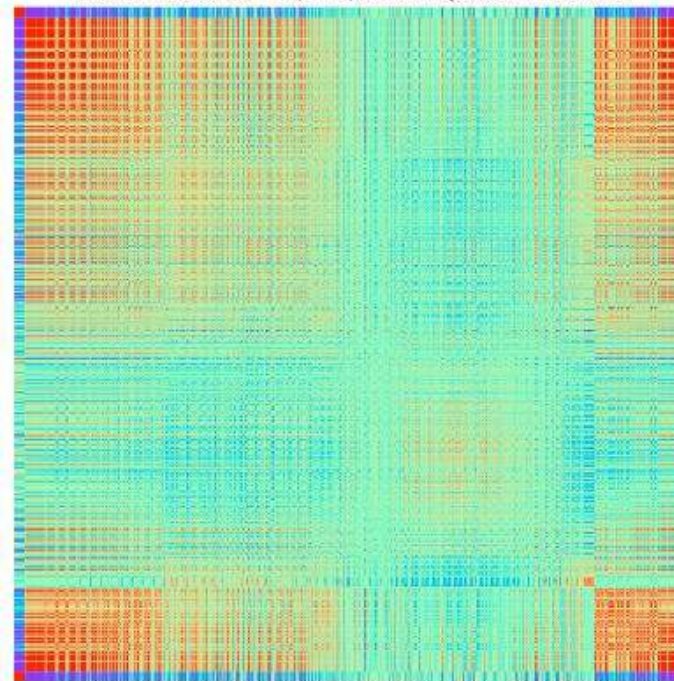


Red Hopfield

Antes de pasar a una implementación de HN en Python, hay dos conceptos a nivel de red que van de la mano y que es útil comprender:

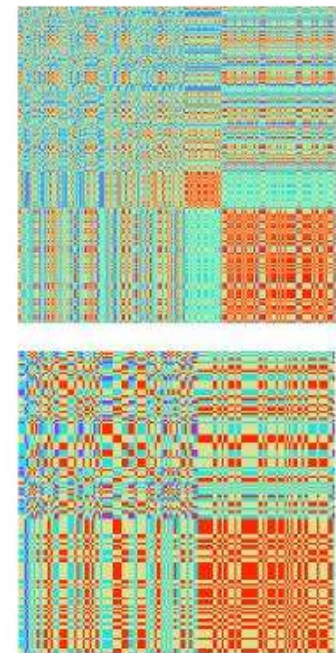
- **Aprendizaje en red:** el cálculo de pesos de red que permitirán que el estado de la red evolucione hacia uno o más atractores de memoria.
- **Energía de red:** una base matemática y una analogía útil que ayuda a comprender por qué se garantiza que la red convergerá a un estado atractor o mínimo energético.
- Primero, echemos un vistazo a la matriz de pesos de la red de 4096.

4096 Neurons - 16,777,216 unique connections



Each row/column represents a neuron, each square a connection

zoomed-in





Red Hopfield

- Lo primero que debes notar es que este gráfico se ve muy bien. Cada fila/columna representa una neurona, y las intersecciones entre filas/columnas representan el peso de la conexión codificado por colores.
- Lo segundo que debes notar es que tenemos una matriz diagonalmente simétrica. Esto es de esperar, ya que las intersecciones coloreadas entre filas/columnas representan los pesos de las conexiones simétricas y bidireccionales que mencionamos anteriormente.
- Técnicamente, solo necesitamos la mitad de este cuadrado, pero una matriz simétrica como la de arriba es más fácil de calcular e implementar en una computadora, y es mucho más atractiva.
- Ahora sabemos que los pesos de las conexiones de la red determinan la dinámica de sus atractores. Así que la pregunta clave es: ¿cómo diseñamos estos pesos para que la red tenga un atractor para cada recuerdo que deseamos que recuerde?
- Existen varios algoritmos de aprendizaje conocidos por su utilidad para las redes neuronales humanas (HN), pero en este caso utilizaremos el aprendizaje hebbiano. El algoritmo de aprendizaje hebbiano se originó como una regla para describir el fenómeno de la plasticidad sináptica en las redes neuronales biológicas. En general, el aprendizaje hebbiano puede resumirse en la afirmación de que «las neuronas que se activan juntas, se conectan entre sí». En otras palabras, si dos neuronas conectadas mediante sinapsis están activas simultáneamente, la fuerza de su conexión aumenta (en el caso de las HN, es más positiva) y su actividad se correlaciona más con el tiempo.





Red Hopfield

- Lo contrario ocurre con las neuronas que se activan asincrónicamente; en cambio, sus pesos de conexión disminuyen (en el caso de las neuronas hebbianas, se vuelven más negativos). En nuestro ejemplo de imágenes almacenadas, la intuición detrás del aprendizaje hebbiano es clara. Buscamos que el peso entre las neuronas que están simultáneamente "activadas" o "apagadas" en el patrón objetivo fomente la alineación de su actividad a medida que la red se actualiza. También buscamos que las células cuyos estados se oponen tengan el comportamiento opuesto.
- La fórmula para calcular un peso único entre dos neuronas, i y j , según el aprendizaje hebbiano viene dada por:

$$w_{ij} = s_i s_j$$

- Donde s_i y s_j son los estados respectivos que adoptarán las neuronas i y j en el patrón objetivo. Para calcular los pesos que resultan en varios patrones almacenados simultáneamente, la fórmula se expande...:

$$w_{ij} = \frac{1}{n} \sum_p s_i^p s_j^p = \frac{1}{n} Y Y^T$$



Red Hopfield

- Que efectivamente toma el peso promedio calculado entre los patrones, donde p representa cualquier patrón y n el número de patrones. A la derecha, tenemos la expresión equivalente en forma matricial, donde Y es una matriz que contiene nuestros patrones como vectores fila. Es importante destacar que, al almacenar múltiples patrones, el número de bits en cada patrón debe mantenerse constante mientras la red aprende mediante este algoritmo.
- Como se mencionó anteriormente, una característica significativa de este tipo de aprendizaje es que es de una sola toma, lo que significa que la red solo requiere una exposición para aprender. Esto se debe a que calculamos la totalidad de la matriz de pesos en una sola operación. El aprendizaje de una sola toma es bastante único, ya que contrasta directamente con el aprendizaje profundo moderno mediante retropropagación ; un proceso caracterizado por ajustar iterativamente los pesos de la red para reducir una señal de error. Dependiendo de la tarea, el entrenamiento mediante retropropagación a menudo requiere decenas de miles, o incluso millones, de ejemplos de entrenamiento. Esto lo convierte en un candidato pobre como modelo para el procesamiento de la memoria episódica humana que, por definición, requiere un episodio para la memorización. Curiosamente, el hipocampo, un área conocida por tener dinámicas similares a las de los atractores, se recluta activamente durante el aprendizaje de una sola toma , y se sabe que los conjuntos dispersos en esta área coordinan la recapitulación de estados corticales pasados durante la recuperación de la memoria episódica .





Red Hopfield

- Hasta ahora hemos abordado la estructura, el comportamiento y el aprendizaje de las redes neuronales. A continuación, conectaremos todo y comprenderemos por qué las redes neuronales se comportan como lo hacen utilizando el concepto de energía de red.
- Sabemos que, independientemente del estado inicial de una red neuronal, esta tiende hacia un atractor de "punto fijo" en el espacio de estados. "Punto fijo" es una forma de describir que el estado de la red permanece estable; es decir, la red podría actualizarse indefinidamente en este punto y ninguna neurona se invertiría. Este fenómeno no nos es ajeno; observamos con frecuencia sistemas que convergen a estados estacionarios.
- Consideremos una pelota rodando cuesta abajo hasta que se asienta en el fondo, o una moneda que se deja caer girando rápidamente en el suelo hasta que se queda estacionaria sobre uno de sus lados. Los estados de estos sistemas, que podrían describirse como las posiciones, velocidades, etc. de los objetos, convergen a configuraciones que no cambian sin perturbaciones adicionales. Utilizando el modelo de Ising como ejemplo, las orientaciones de los dipolos magnéticos en un material magnético de alta temperatura podrían estar en un estado altamente entrópico y desordenado.





Red Hopfield

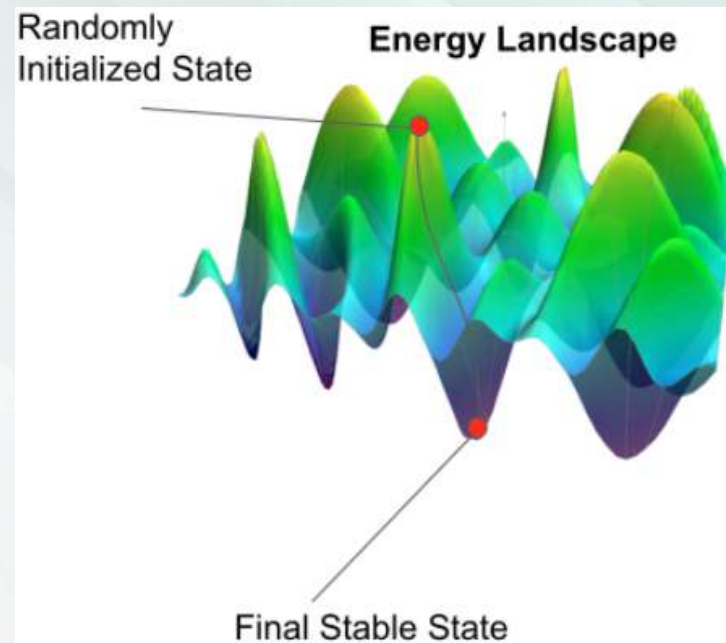
- Si se permitiera que el material se enfriara, los dipolos se alinearían independientemente con sus campos locales, y todo el material tendería a un estado homogéneo y estable, alineado globalmente. La característica común de estos ejemplos es que sus estados finales se encuentran en mínimos energéticos. En otras palabras, es termodinámicamente favorable que alcancen el reposo, por lo que, desde el momento en que cesan las perturbaciones externas de cada sistema, comienzan sus respectivas trayectorias de estado hacia la energía más baja posible.
- Las redes hiperbáricas implementadas en la computadora no son sistemas físicos que evolucionen hacia estados termodinámicamente estables. Sin embargo, debido a las restricciones inherentes a la red, definidas por su estructura y reglas de actualización, muestran un comportamiento análogo. Cabe recordar que inicialmente se inspiraron en el modelo de Ising.
- De hecho, las HN tienen un valor escalar, denominado Energía , que disminuye monótonamente con cada generación de actualización posterior. La trayectoria de estado de una HN a lo largo del tiempo puede considerarse como un punto descendente en un paisaje energético; cuando la red converge a una memoria, se encuentra en un «valle» del paisaje y no evolucionará más a menos que se altere.





Red Hopfield

- La ventaja de los HN reside en que, dependiendo de los pesos aprendidos, los respectivos atractores, o mínimos energéticos, son ajustables. Cualquier estado del sistema puede ser , o aprenderse , un atractor, y, por lo tanto, cualquier estado puede ser una memoria aprendida. A continuación, se presenta una descripción matemática de la energía en los HN, que muestra cómo se garantiza que esta disminuirá en las siguientes generaciones de actualizaciones.





INSTALACIÓN DE LIBRERÍAS

Instalación de librerías principales (opencv-python, numpy, matplotlib, tqdm)
pip install opencv-python numpy matplotlib tqdm

```
Microsoft Windows [Versión 10.0.26100.4652]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\norbi>pip install opencv-python numpy matplotlib tqdm
Requirement already satisfied: opencv-python in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages
(4.9.0.80)
Requirement already satisfied: numpy in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages (1.26.3)
Requirement already satisfied: matplotlib in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages (3
.10.1)
Requirement already satisfied: tqdm in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages (4.66.4)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packa
ges (from matplotlib) (1.2.0)
Requirement already satisfied: cycler>=0.10 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages
(from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-pack
ages (from matplotlib) (4.47.2)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-pack
ages (from matplotlib) (1.4.5)
Requirement already satisfied: packaging>=20.0 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packag
es (from matplotlib) (23.2)
Requirement already satisfied: pillow>=8 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages (fr
om matplotlib) (10.2.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packa
ges (from matplotlib) (3.1.1)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-p
ackages (from matplotlib) (2.8.2)
Requirement already satisfied: colorama in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages (fro
m tqdm) (0.4.6)
Requirement already satisfied: six>=1.5 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages (fro
m python-dateutil>=2.7->matplotlib) (1.16.0)

[notice] A new release of pip is available: 25.0.1 -> 25.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\Users\norbi>
```




INSTALACIÓN DE LIBRERÍAS

Instalación de librerías complementarias (tensorflow)
pip install tensorflow

```
Microsoft Windows [Versión 10.0.26100.4652]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\norbi>pip install opencv-python numpy matplotlib tqdm
Requirement already satisfied: opencv-python in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages
(4.9.0.80)
Requirement already satisfied: numpy in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages (1.26.3)
Requirement already satisfied: matplotlib in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages (3
.10.1)
Requirement already satisfied: tqdm in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages (4.66.4)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packa
ges (from matplotlib) (1.2.0)
Requirement already satisfied: cycler>=0.10 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages
(from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-pack
ages (from matplotlib) (4.47.2)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-pack
ages (from matplotlib) (1.4.5)
Requirement already satisfied: packaging>=20.0 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packag
es (from matplotlib) (23.2)
Requirement already satisfied: pillow>=8 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages (fr
om matplotlib) (10.2.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packa
ges (from matplotlib) (3.1.1)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-p
ackages (from matplotlib) (2.8.2)
Requirement already satisfied: colorama in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages (fro
m tqdm) (0.4.6)
Requirement already satisfied: six>=1.5 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages (fro
m python-dateutil>=2.7->matplotlib) (1.16.0)

[notice] A new release of pip is available: 25.0.1 -> 25.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\Users\norbi>
```



INSTALACIÓN DE LIBRERÍAS para yolo

Instalación de librerías YOLO(ultralytics): **pip install ultralytics**

```
Microsoft Windows [Versión 10.0.26100.4652]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\norbi>pip install opencv-python numpy matplotlib tqdm
Requirement already satisfied: opencv-python in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages
(4.9.0.80)
Requirement already satisfied: numpy in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages (1.26.3)
Requirement already satisfied: matplotlib in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages (3
.10.1)
Requirement already satisfied: tqdm in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages (4.66.4)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packa
ges (from matplotlib) (1.2.0)
Requirement already satisfied: cycler>=0.10 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages
(from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-pack
ages (from matplotlib) (4.47.2)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-pack
ages (from matplotlib) (1.4.5)
Requirement already satisfied: packaging>=20.0 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packag
es (from matplotlib) (23.2)
Requirement already satisfied: pillow>=8 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages (fr
om matplotlib) (10.2.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packa
ges (from matplotlib) (3.1.1)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-p
ackages (from matplotlib) (2.8.2)
Requirement already satisfied: colorama in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages (fro
m tqdm) (0.4.6)
Requirement already satisfied: six>=1.5 in c:\users\norbi\appdata\local\programs\python\python310\lib\site-packages (fro
m python-dateutil>=2.7->matplotlib) (1.16.0)

[notice] A new release of pip is available: 25.0.1 -> 25.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip

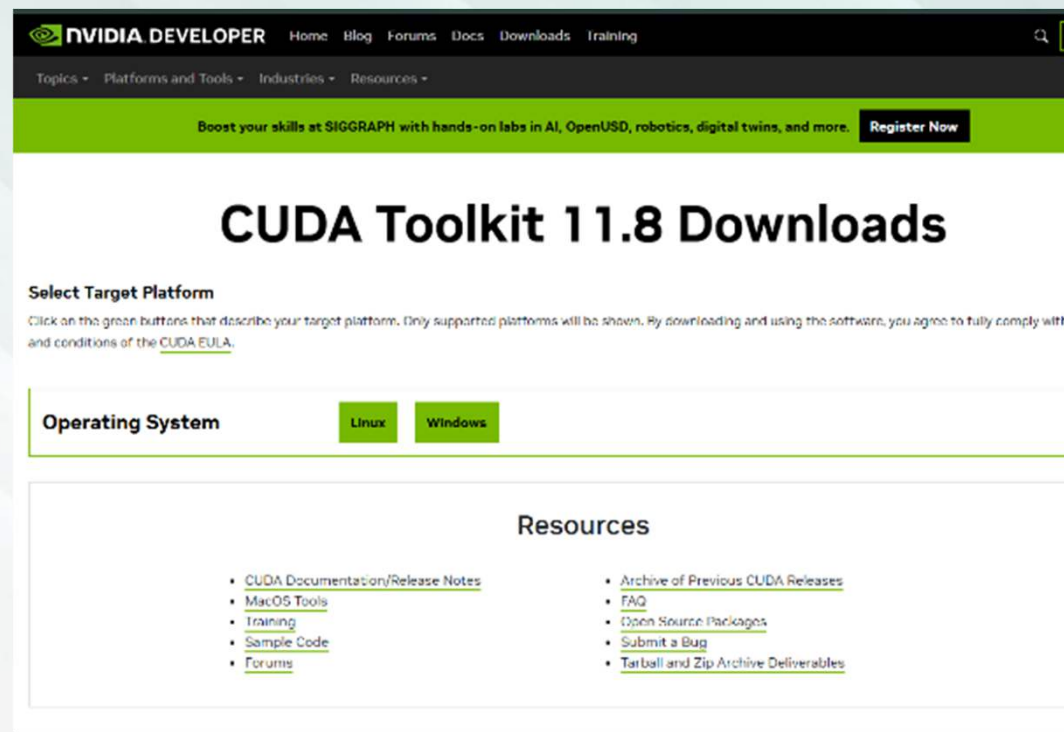
C:\Users\norbi>
```



INSTALACIÓN DE CUDA para tarjeta gráfica (nvidia)

Instalación de CUDA Toolkits: <https://developer.nvidia.com/cuda-11-8-0-download-archive>

En caso de contar con tarjeta gráfica de NVIDIA





INSTALACIÓN DE LIBRERÍAS para pytorch

Ingresamos a la siguiente página: <https://pytorch.org/get-started/locally/>

PyTorch

Learn Community Projects Docs Blog & News About JOIN

or get started quickly with one of the supported cloud platforms.

Start Locally PyTorch 2.x Start via Cloud Partners Previous PyTorch Versions PyTorch for Edge

Shortcuts

- Prerequisites
 - Supported Windows
 - Distributions
 - Python
 - Package Manager
- Installation
 - pip
- Verification
- Building from source
- Prerequisites

Start Locally

Select your preferences and run the install command. Stable represents the most currently tested and supported version of PyTorch. This should be suitable for many users. Preview is available if you want the latest, not fully tested and supported, builds that are generated nightly. Please ensure that you have met the prerequisites below (e.g., numpy), depending on your package manager. You can also install previous versions of PyTorch. Note that LibTorch is only available for C++.

NOTE: Latest PyTorch requires Python 3.9 or later.

PyTorch Build	Stable (2.7.1)	Preview (Nightly)
Your OS	Linux	Mac Windows
Package	Pip	LibTorch Source
Language	Python	C++ / Java
Compute Platform	CUDA 11.8	CUDA 12.6
	CUDA 12.6	ROCM6-8 CPU

Run this Command:

```
pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118
```




**UNIVERSIDAD
CATÓLICA**
SEDES SAPIENTIAE